

OIFC 7.4 模拟七

OIFC 未来共同体

题目名称	物质鉴定	对拍	错误
题目类型	交互题型	传统题型	传统题型
目录	/	sam	bug
可执行文件名	/	sam	bug
输入文件名	/	sam.in	bug.in
输出文件名	/	sam.out	bug.out
每个测试点时限	1 秒	1 秒	2 秒
内存限制	256 MB	512 MB	512 MB
测试点数目	10	20	20
测试点是否等分	是	是	是

编译选项

对于 C++ 语言	-lm -O2 -std=c++14 (已 c++14 为例)
-----------	---------------------------------

注意事项

1. 文件名（包括程序名，后缀名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须为 0。
3. 提交的程序代码文件的放置位置请参照考场具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 程序可使用的栈内存空间限制与题目的内存限制一致。
7. 评测在 xyd 评测机下进行。
8. 最终评测时所用的编译命令中不含编译选项之外的任何优化开关。
9. oj 的单题代码长度限制好像是 50KB 还是 64KB 来着，请注意不要爆了。

物质鉴定 (identify)

【题目描述】

问题描述

这是一道交互题

小 H 在化学实验室做实验课。

现在实验室中有 N 瓶液体, 编号为 $[1, N]$, 小 H 知道每瓶液体可能是 0.1mol/L 的 NaOH 溶液或 HCl 溶液。现在它要鉴定总共有多少瓶 NaOH 溶液。

实验室中有许多 PH 试纸, 不过由于比色卡丢失, 这些 PH 试纸只能鉴定溶液为酸性还是碱性。小 H 每次可以从一瓶溶液中取出 10ml 溶液, 然后将他们混合, 最后用 PH 试纸鉴定结果。由于实验室的特殊规定, 每次混合溶液时不得从一瓶溶液中取多次液体。换句话说, 每次混合溶液为选取一个不可重集 S , 并从属于集合 S 的瓶子中取出 10ml 溶液混合。

不过小 H 不仅是一名初三学生, 它还是一名大魔法师。它可以使用托耶雷特魔咒将区间 $[l, r]$ 内的所有溶液的类型翻转。不过由于魔咒使用机制, 对于每一瓶溶液, 它只能对其翻转一次。

由于 PH 试纸很贵, 小 H 希望它使用最少的试纸得出最开始时的答案。它去询问小 x 怎么做, 但是小 x 一眼秒了这题, 而他还要去完成每日委托, 所以他把问题给了你。

提示: NaOH 为碱性, HCl 为酸性, 1mol NaOH 与 1mol HCl 反应产物为中性。

评测方式

这是一道交互题, 你的程序不需要也不能包含主函数, 而你的程序也不需要进行输入输出, 为了你的程序能正常编译, 请务必注意以下内容

你需要提交一个名称为 `identify.cpp` 的程序。

交互库提供以下四个可调用函数:

`int size();` 返回值为 N

`int test(vector<int> x);` 数组 x 为你要查询的下标数组, 而返回值表示这些溶液混合后的结果。 -1 为酸性, 0 为中性, 而 1 为碱性。

请注意, 交互库一次查询的复杂度为 $\mathcal{O}(|x|)$, 请在实现时注意你的运行速度。

另外, 如果你的 x 的大小大于 10^6 , 你将会在这个测试点获得 0 分

`void Reverse(int l, int r)` 翻转下标为 $[l, r]$ 区间内的所有溶液类型。如果你重复翻转了一个元素, 你将获得 0 分。

`void answer(int x);` 用于提交你的结果。 x 表示 NaOH 溶液的数量。调用本函数后会强制结束你的程序。

你需要实现一个函数 `solve()`, 交互库会调用仅一次本函数, 你需要在这个函数中实现题目的全部功能, 也就是通过询问交互库得到正确答案并通过 `answer` 函数提交你的结果。

为了你的程序能正常编译, 请在你的程序中定义以上五个函数, 以下是一个可正常编译的程序例子:

```
#include<bits/stdc++.h>
```

```

int size();
int test(std :: vector<int > d);
void Reverse(int l,int r);
void answer(int x);
//请务必注意这些函数的定义是必要的，且需要在被访问前定义
void solve(){
    int n = size();//输入 n
    if(n == 2){//在n = 2时得出正确答案
        std::vector<int > a,b;
        a.push_back(1),b.push_back(2);
        int q1 = test(a),q2 = test(b);
        int cnt = 0;
        if(q1 == 1)cnt++;
        if(q2 == 1)cnt++;
        answer(cnt);
    }
    else answer(n);//认为全都是NaOH,显然不一定可以得到正确结果
}

```

评分方法

如果你没有得到正确答案，你会获得 0 分。

如果你获得正确答案，令 *yourans* 为你的程序调用 `test(vector<int > x)` 的次数，*stdans* 为标准使用次数。

若 $yourans \leq stdans$ ，你获得这个测试点 100% 的分数

否则，你获得 $\max(0, 100 - \frac{(yourans - stdans)}{100})$ 的分数

为了方便你在本机调试，以下是一个交互库的实现：

```

#include<bits/stdc++.h>
using namespace std;
const int MX = 1e5 + 7,Max_sz = 1e6;
int col[MX],a[MX],hard,testcase = 0,N;
bool mark[MX];
void gen(){
    cin >> N;hard = 1;
    for(int i = 1;i <= N;i++)cin >> col[i];
    for(int i = 1;i <= N;i++)a[i] = col[i];
}
int size(){return N;}
int test(vector<int > d){

```

```
if(d.size() > Max_sz){
    cerr << "Query is too big\n";
    exit(0);
}
set<int > s;
int cnt = 0;
for(int i = 0;i < d.size();i++){
    int x = d[i];
    if(!(x >= 1 && x <= N)){
        cerr << x << " out of range\n";
        exit(0);
    }
    if(s.find(x) == s.end() || !hard)cnt += a[x];
    else{
        cerr << "Used querys!\n";
        exit(0);
    }
    s.insert(x);
}
testcase++;
if(cnt > 0)return 1;
else if(cnt == 0)return 0;
else return -1;
}

void Reverse(int l,int r){
    for(int i = l;i <= r;i++){
        if(mark[i]){
            cerr << i << " already reversed\n";
            exit(0);
        }
        mark[i] = true;
        a[i] = -col[i];
    }
}

void answer(int x){
    int cnt = 0;
    for(int i = 1;i <= N;i++){
        if(col[i] == 1)cnt++;
    }
    if(cnt == x){
```

```

        cerr << "Correct,testcase = " << testcase << '\n';
        exit(0);
    }
    else{
        cerr << "Incorrect,your answer = " << x << " real answer = "
            << cnt << '\n';
        exit(0);
    }
}
}
void solve();
int main(){
    gen();
    solve();
}

```

不保证该的交互库与真实交互库完全一致。

评测用例规模与约定

对于前 30% 的数据，满足 $N = 5000$ ，标准答案 $stdans = 500$

对于后 70% 的数据，满足 $N = 10^5$ ，标准答案 $stdans = 2500$

如何使用下发的交互库

将交互库程序与你的程序放入同一个文件夹中，将交互库与你的程序一起编译，即使用以下命令：

```
g++ grader.cpp identify.cpp -o identify
```

其中，grader.cpp 为交互库，identify.cpp 为你的程序，identify 为生成的可执行文件名称。

交互库会从标准输入中输入两行数，表示溶液的初始状态，具体方法如下：

第一行，输入一个数 N ，表示溶液数量

第二行，输入 N 个数，第 i 个数表示第 i 种溶液的类型， -1 为酸性， 1 为碱性。

如果你的程序正常结束并得到了正确的结果，交互库会输出一行：

Correct,testcase = X ，其中 X 为你使用 `test(vector<int> x)` 的次数

否则根据你的情况输出对应结果：

Query is too big 你调用 `test(vector<int> x)` 时 x 的大小过大 (超过 10^6)。X out of range 你调用 `test(vector<int> x)` 时候 x 中有元素超出 $[1, N]$ 的范围。Used queries! 你调用 `test(vector<int> x)` 时候 x 中有重复元素。already reversed 你在调用 `void Reverse(int l,int r)` 时对某个溶液使用了多次魔咒。"Incorrect,your answer = x real answer = y 你的答案与真实答案不符，你的答案为 x ，真实答案为 y 。

以下是一个样例：

交互库	你的程序	解释
读入 5		
读入 1 -1 -1 1 1		
调用 <code>solve()</code>		
	调用 <code>size()</code>	
返回 5		返回总溶液数量
	调用 <code>test({1,2,3})</code>	
返回 -1		三瓶溶液中酸性偏多，中和结果为酸性
	调用 <code>Reverse(2,4)</code>	此时溶液情况为: 1 1 1 -1 1
	调用 <code>test({2,4,5})</code>	
返回 1		
	调用 <code>answer(3)</code>	
输出 <code>Correct, testcase = 2</code>		注意你需要回答的是开始时的情况

【输入格式】

本题没有输入。

【输出格式】

本题没有输出

【样例输入】

Input 1

【样例输出】

Output 1

【样例解释】**【数据范围与提示】****题目附件**

identify.zip

【更多提示】

下发样例与真实数据使用同一个 Generator 和基本一致的参数生成，你可以用下发样例来估计评测数据的实际范围。

请不要卡评测。

对拍 (sam)

【题目描述】

1s 512 MB

文件输入输出名 `sam.in/sam.out`

ChiFAN 太菜了！他随手点开一道题目，写完之后发现过了样例却喜提零昏，所以你需要帮他写个程序对拍。

具体来说，给出 n 个字符串 $s_1 \rightarrow s_n$ ，我们定义函数 $F(s, t)$ 与函数 $G(s, t)$ 分别表示字符串 s, t 最长公共前缀与最长公共后缀的长度，你要求出：

$$\sum_{i=1}^n \sum_{j=1}^{i-1} F(s_i, s_j) \times G(s_i, s_j)$$

由于答案可能很大，所以你只要求出答案对 $2^{32} - 1$ 取模后的结果。

当然对拍不可能只拍一组，所以一个测试点内有多组测试数据。

【输入格式】

第一行一个数 T 表示数据组数。

接下来对于每组数据，第一行一个数 n 表示字符串数量，接下来 n 行每行一个字符串。

【输出格式】

对于每组测试数据输出一行一个数表示答案。

【样例输入】

Input 1

```
1
4
genshin
generalsandgenshin
genshinseeyouagain
genneralsagain
```

Input 2


```
1
5
xiezheyuan
ytxy
chifanxzy
chiytandxzy
genshin
```

Input 3

见下发 samex.in

【样例输出】

Output 1

68

Output 2

9

Output 3

见下发 samex.ans

【样例解释】

样例 1 解释

我们以 (u, v) 表示第 u 个字符串与第 v 个字符串的产生的贡献

$$(1, 2) = 3 \times 7 = 21$$

$$(1, 3) = 7 \times 2 = 14$$

$$(1, 4) = 3 \times 2 = 6$$

$$(2, 3) = 3 \times 2 = 6$$

$$(2, 4) = 3 \times 2 = 6$$

$$(3, 4) = 3 \times 5 = 15$$

$$21 + 14 + 6 + 6 + 6 + 15 = 68$$

【数据范围与提示】

我们记单组测试数据内 $\sum |s_i|$ 为 len 。

对于 10% 的数据， $\max(len) \leq 100, T = 1$ 。

对于另外 10% 的数据，保证 s_i 为回文串。

对于另外 15% 的数据， $T = 1$ 。

对于另外 15% 的数据， $T \leq 10$ 。

对于另外 20% 的数据，保证出现的字符串中只包含字符 **a** 或者 **b**。

对于 100% 的数据，保证 $\max(len) \leq 10^5, T \leq 10^2$ 且出现的字符全部为英文小写字母。

题目附件

sam.zip

【更多提示】

下发样例与真实数据使用同一个 Generator 和基本一致的参数生成，你可以用下发样例来估计评测数据的实际范围。

请不要卡评测。

错误 (bug)

【题目描述】

2s 512 MB

文件输入输出名 `bug.in/bug.out`

Qiuly 代码习惯不检点，写的复杂无比，还容易出 bug。

Qiuly 的 DP 代码中有 n 个数组，第 u 个数组的大小为 s_u ，下标 $0 \cdots s_u - 1$ ，我们称数组 u 的第 i 个位置为 $a_{u,i}$ 。经过带 bug 的预处理后，其中有些元素的值是错误的，这种错误被称为初级错误。

这 n 个数组中每两个数组间都有单向转移关系，也即形成一张竞赛图。在预处理完成后进入 DP 部分，DP 过程中，第 t 轮会发生以下操作：对于每个转移关系 $u \rightarrow v$ ，如果 $a_{v,t \bmod s_v}$ 没有错误且 $a_{u,t \bmod s_u}$ 有错误，则 $a_{v,t \bmod s_v}$ 会产生次级错误。同一轮中的所有操作是同时进行的，不会相互影响。DP 会进行足够多轮，使得最后不会产生新的错误。

运行结束后，Qiuly 会开始 debug，标记所有找到的错误。具体而言，每个初级错误的位置都会被找到，而经过 DP 转移仅出现次级错误的位置可能会被找到也可能不会，而其他既没有初级错误，又在 DP 过程中没有转移出次级错误的位置则一定不会被认为有错误。现在，Qiuly 想知道，在所有情况中，从标记出错误最多的 A 个数组中选出 B 个数组，能选出多少种不同的数组集合。

注意 1：如果两个数组找出的错误数量相同，你可以任意钦定他们的顺序。例如，如果三个数组找出的错误数量分别为 1, 1, 2，那么找出错误最多的两个数组可以是第一个和第三个，也可以是第二个和第三个。

注意 2：次级错误和初级错误都算是错误，它们都可以转移出新的次级错误。

【输入格式】

第一行三个整数 n, A, B 。

接下来 n 行每行一个长为 n 的 01 串，第 i 行第 j 个为 $g_{i,j}$ ，如果 $g_{i,j} = 1$ 表示有数组 i 到数组 j 的转移关系。

接下来 n 行，每行一个整数 s_i 和一个长为 s_i 的 01 串，表示初始化后数组 i 每个元素是否有初级错误，1 为有，0 为无。

【输出格式】

一行一个整数表示能选出的不同数组集合数量。

【样例输入】

Input 1

```
5 2 1
00000
10000
11000
11100
11110
2 01
1 1
1 0
1 1
6 100100
```

【样例输出】

Output 1

5

【样例解释】

见下发 `sample.zip` 压缩包。`0.in/out` 为小样例, 不保证满足任何性质。其余标号 1/3/5/9/13/17 的样例均与标号相同的实际测试点相同限制。

【数据范围与提示】

总共 20 个测试点, 测试点等分。

对于所有测试点满足: $n \leq 5 \cdot 10^3$, $\sum s_u \leq 2 \cdot 10^6$, 保证给出的转移关系是一张竞赛图, 也即 $g_{u,u} = 0, \forall u \neq v, g_{u,v} + g_{v,u} = 1$ 。

测试点 1, 2 满足 $n \leq 10, s_u \leq 10$ 。

测试点 3, 4 满足对于任意一个数组, 要么所有元素都有初级错误, 要么所有元素都没有初级错误。

测试点 5, 6, 7, 8 满足所有 s_u 相等。

测试点 9, 10, 11, 12 满足图随机生成。

测试点 13, 14, 15, 16 满足 $n \leq 500, \sum s_u \leq 10^5$

测试点 17, 18, 19, 20 没有特殊限制。

题目附件

`sample.zip`

【更多提示】

下发样例与真实数据使用同一个 Generator 和基本一致的参数生成，你可以用下发样例来估计评测数据的实际范围。

请不要卡评测。